C Programming

Functions: Ch.8-9

Concept & modularization | Parameters & return values | Scope | Recursion

What You Will Learn in This Chapter

- Modularization
- Concept of **function**
- How to write a **function**
- **Return value**, transfer of **arguments**
- Why use functions? Large programs should be written by breaking down the overall problem into simpler, more understandable functions.

Ch. 8

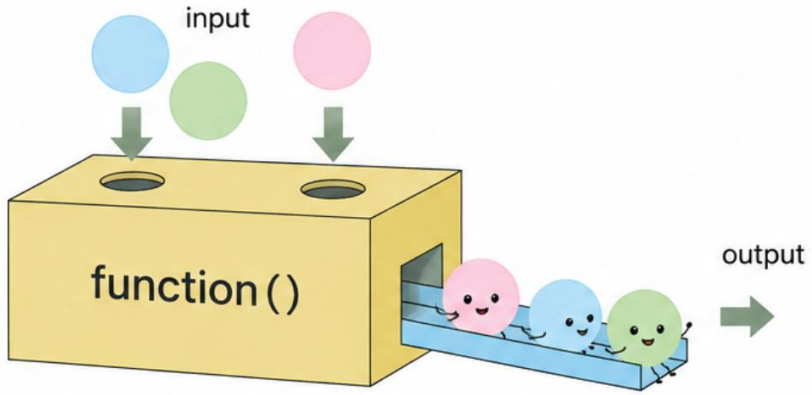
Functions

Concept of Function

- **Function**: it is like a black box that takes input, performs a specific task, and returns a result.

- **Function**

  A function is a block of code that performs a specific task.



- **Why use a function?**

Reusability
Write once, use many times.

Modularity
Break a large program into small, manageable pieces.

Readability & Maintenance
Code is easier to read, test, and update.

- **Example**

Function Call (use)

```
print_stars();
print_stars();
```

Function Definition

```
void print_stars()
{
    for (int i = 0; i < 30; i++)
        printf("*");
}
```

Define the function once, then call it whenever you need it.

Type of Function

- **User-defined functions**

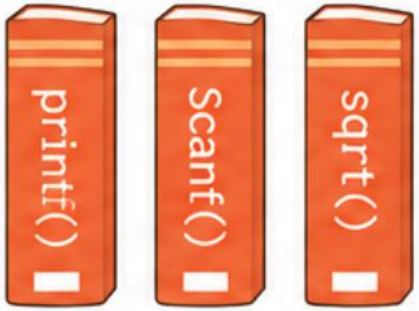
  Functions created by the programmer for specific tasks.



Since the function I want isn't available, I have to create it myself!

- **Library functions**

  Predefined functions provided by the C standard libraries.



These functions are provided **by default**.

Key Point

If a function you need is not available, create your own. Otherwise, use library functions.

Function Definition

1. Function Definition Syntax

Return type — Function name — Parameter list (currently none)

```
void print_stars()
{
    for(int i=0; i<30; i++)
        printf("*");
}
```

Function body (the code to be executed)

2. Function Names

```
int     square()            // Function to square an integer
double  compute_average()   // Function to calculate the average
void    set_cursor_type()   // Function to set the type of cursor
```

function name

4. Key Point



If a function you need is not available,
create your own.
Otherwise, use library functions.

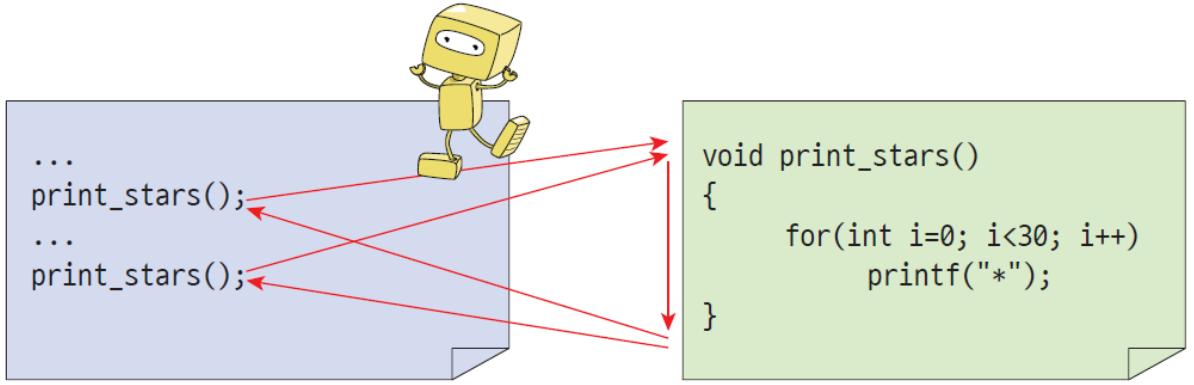


Is there a limit to the length of a function?

In C, there is no strict limit on the length of a function. This means that you can write a lot of statements in a single function.
However, if a function becomes too long, it is not good. Ideally, a function should do one specific task.
If a function is too long, it may be performing more than one task. In that case, it is better to break it down into smaller functions.
While there is no absolute standard, keeping functions under about 30 lines is a good guideline.

Summary: Features and Pros of Functions

- A **function** is a collection of instructions to perform a specific task; functions have distinct names, perform a specific task, and can take input and return a result.
- Using functions can help prevent duplication of code.
- Once a **function** is written, it can be reused multiple times.
- Using functions allows you to divide your entire program into modules, making development easier, more organized, and easier to maintain.
- A function call is writing the name of a function, such as print_stars( ).
  - When the execution of the called function is finished, it returns to the calling location and resumes execution of the code that was paused.



Function Call: print_stars()

```c
#include < stdio.h >

void print_stars ()
{
    for ( int i = 0; i < 30; i++)
        printf ( "*" );
}

int main( void )
{
    print_stars ();
    printf ( "\nHello World!\n" );
    print_stars ();
    printf ( "\n" );
    return 0;
}
```

```
******************************
Hello World!
******************************
```

A function call is writing the name of a function, such as print_stars(). Statements inside a function are not executed at all until it is called.

Parameters & Return Values

1. Structure (Syntax)

Return type | Function nam | Parameters

Example

```
int max(int x, int y)
{
    if ( x > y )
        return x;
    else
        return y;
}
```

Function body (the code to be executed)

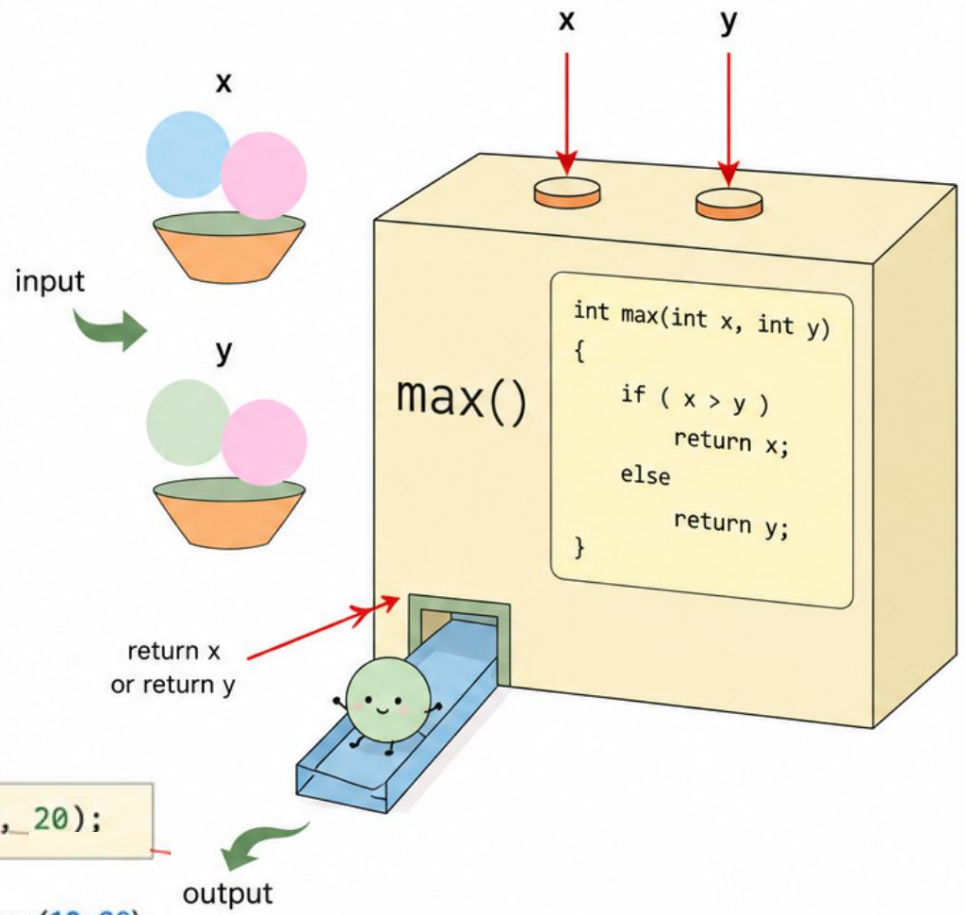
What it means

- `max()` is a function that takes two integers x and y.
- It compares x and y.
- If x is greater, it `returns` x.
- Otherwise, it `returns` y.

```
value = max(10, 20);
```

The return value of max(10, 20) is assigned to variable 'value'.

2. Concept (How it works)

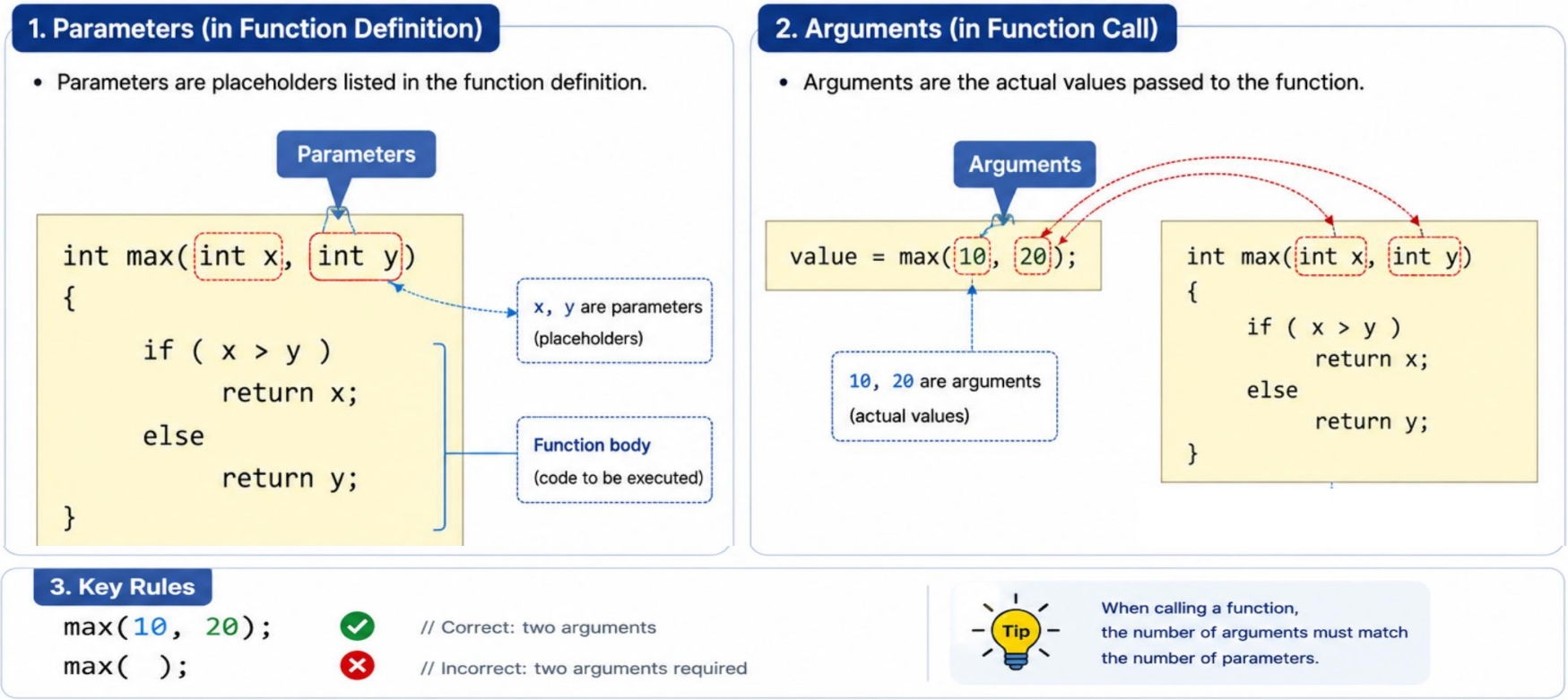


Tip

Functions help organize code, reuse logic, and make programs easier to understand and maintain.

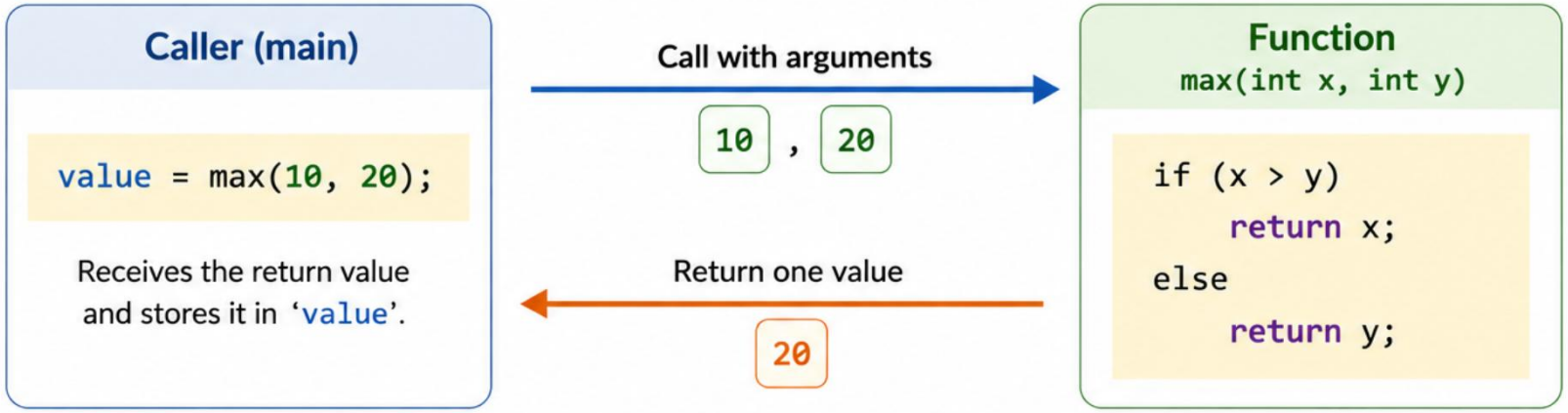
Arguments and Parameters

- A **parameter** is a variable that receives this value.
- An **argument** is a value actually passed to a **function** by the calling program.
- The number of parameters must match exactly - if the number of parameters and arguments do not match, an error will occur.



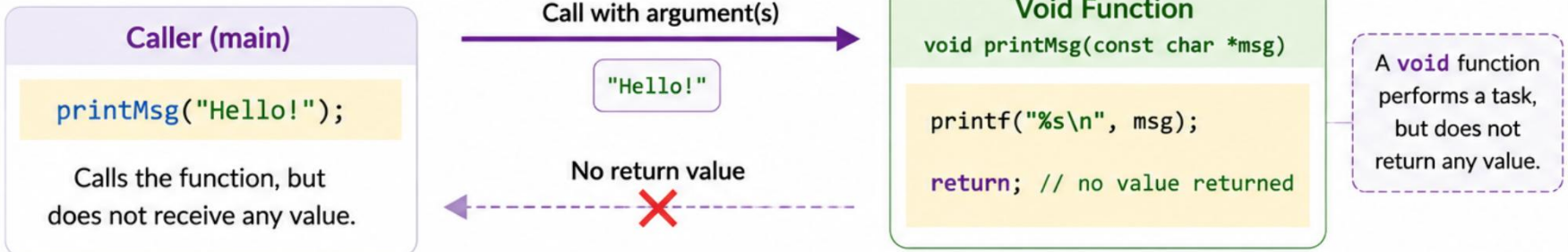
Return Value

1. The **return value** is the result of the operation that the **function returns** to the place where it was called.
2. There can be multiple arguments, but only **ONE return value**.
3. A **void** function does not return a value.



If the function is declared as **void**, it does not return any value.

Example: void function (no return value)



Summary: • Functions may take multiple arguments. | • Functions return only **ONE** value. | • **void** functions return nothing.

Example: max() Function

VSC

```c
int max( int x , int y )
{
    if ( x > y )
        return x ;
    else
        return y ;
}

int main( void )
{
    int x, y, larger;
    printf ( "Enter two integers: " ) ;
    scanf( "%d %d" , &x, &y);

    larger = max(x, y);
    printf ( "The larger value is %d.\n" , larger);
    return 0;
}
```

max() finds the larger value among the values entered by the user.

Combining Functions: Two More Building-Block Functions

- Function: get_integer()
  - a **function** that prints an input guidance message, takes an integer as input, and returns it to us.
- Function: add()
  - a **function** that takes two integers and calculates their sum.
- These small, single-purpose functions can then be combined to build bigger programs.

```c
int get_integer ()
{
    int value;
    printf ( "Enter an integer: " );
    scanf ( "%d" , &value);
    return value;
}

int add( int x , int y )
{
    return x + y ;
}

int main( void )
{
    int x = get_integer ();
    int y = get_integer ();
    int sum = add(x, y);
    printf ( "The sum of the two numbers is %d.\n" , sum);
    return 0;
}
```

Functions Prototype

- A function prototype tells the compiler the function's name, return type, and parameters **before it is used**.
- It is the function header followed by a semicolon (;). Parameter names are optional; only the data types are required.

1. Function Definition (No Prototype)

- The compiler sees the function after it is used.

```c
#include <stdio.h>

int main(void)
{
    printf("Celsius is %lf and Fahrenheit is %lf . \n",
         36.0, c_to_f(36.0));
    return 0;
}

double c_to_f(double c_temp)
{
    return 9.0 / 5.0 * c_temp + 32;
}
```



It works because C (before C99) allows implicit declaration, but this is **not recommended**.

2. Function Prototype

- The prototype tells the compiler about the function before it is used.

```c
#include <stdio.h>

double c_to_f(double c_temp);  // Prototype

int main(void)
{
    printf("Celsius is %lf and Fahrenheit is %lf . \n",
         36.0, c_to_f(36.0));
    return 0;
}

double c_to_f(double c_temp)
{
    return 9.0 / 5.0 * c_temp + 32;
}
```



Recommended: The compiler knows the function signature before it is called.

3. Parameter Name May Be Omitted

- In a prototype, parameter names are optional, but the types and order are required.

```c
double c_to_f(double);        // OK

double c_to_f(double x);      // OK

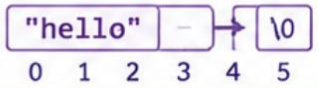
double c_to_f(double c_temp); // OK

int get_integer(void);        // OK
int get_integer();            // OK (same as void)
```

- The parameter name may be **omitted**.
- You must add a **semicolon (;)** at the **end**.

C Standard Library

https://www.w3schools.com/c/c_ref_reference.php

Category	Header File	What It Provides	Common Functions / Examples
1. Standard Input/Output 	stdio.h `#include <stdio.h>`	Functions for input and output, file handling. 	`printf()` – formatted output `scanf()` – formatted input `fopen()` – open a file `fclose()` – close a file
2. Mathematical Operations 	math.h `#include <math.h>`	Mathematical functions and constants. $\sqrt{x}$ x^y π	`sqrt(x)` – square root `pow(x, y)` – x to the power y `sin(x)` – sine (radians) `log(x)` – natural logarithm
3. String Processing 	string.h `#include <string.h>`	Functions to manipulate C strings (character arrays). 	`strlen(s)` – length of string `strcpy(d, s)` – copy string `strcmp(s1, s2)` – compare strings `strcat(d, s)` – concatenate strings
4. Time Handling 	time.h `#include <time.h>`	Functions for date and time management. 	`time(t)` – current time `localtime(t)` – convert to local time `strftime(s, n, f, t)` – format time
5. Error Handling 	errno.h `#include <errno.h>`	Provides error codes and handling utilities.  errno	`errno` – last error code `perror(msg)` – print error message based on errno
6. Searching and Sorting Data 	stdlib.h `#include <stdlib.h>`	Functions for searching, sorting, and general utilities. 	`qsort(base, n, size, cmp)` – sort data `bsearch(key, base, n, size, cmp)` – binary search `rand()` – random number `abs(x)` – absolute value

C Standard Library : <math.h>

https://www.w3schools.com/c/c_ref_math.php

1. What is a Library Function?

- A library function is a pre-written function provided by the C standard library. You can use it by including the proper header file (e.g., `<math.h>`).



2. How to Use

```
#include <math.h>

double y = sin(x);  // call library function
```

Link with math library when compiling:
(e.g.) `gcc program.c -o program -lm`

4. Key Points

- Include the correct header file (e.g., `<math.h>`).
- Use the correct function prototype.
- Pass arguments of the correct type.
- Functions return a value.

3. Common Math Library Functions (<math.h>)

Category	Function Prototype	Explanation
Trigonometry 	`double sin(double x)`	Sine calculation
	`double cos(double x)`	Cosine calculation
	`double tan(double x)`	Calculate tangent value
Inverse Trigonometric Functions 	`double acos(double x)`	Inverse cosine, range is $[0, \pi]$
	`double asin(double x)`	Inverse sine, range is $[-\pi/2, \pi/2]$
	`double atan(double x)`	Inverse tangent, range is $[-\pi/2, \pi/2]$
Hyperbolic Functions 	`double cosh(double x)`	Hyperbolic cosine
	`double sinh(double x)`	Hyperbolic sine
	`double tanh(double x)`	Hyperbolic tangent
Exponential Functions 	`double exp(double x)`	e^x
	`double log(double x)`	$\log_e x$ (natural logarithm)
	`double log10(double x)`	$\log_{10} x$ (base-10 logarithm)
Other Useful Functions $\sqrt{x}$	`double ceil(double x)`	Smallest integer not less than x
	`double floor(double x)`	Largest integer not greater than x
	`double fabs(double x)`	Absolute value of real x
	`int abs(int x)`	Absolute value of integer x
	`double pow(double x, double y)`	x^y (power function)
	`double sqrt(double x)`	$\sqrt{x}$ (square root)

C Standard Library : Generating Lotto Numbers

VSC



Random Numbers in C



Key Concepts

rand()

- Generates a pseudo-random integer.
- Returns a value between 0 and RAND_MAX (32767).

rand() % MAX + 1

- Generates a number from 1 to MAX (inclusive).

srand()

- Sets the starting seed for the random number generator.

srand((unsigned)time(NULL))

- Uses the current time as the seed so you get a different sequence each time the program runs.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX 45

int main(void)
{
    srand((unsigned)time(NULL));
    // set seed with current time

    for (int i = 0; i < 6; i++)
        printf("%d ", rand() % MAX + 1);

    return 0;
}
```

Example Output

```
4 21 22 34 37 38
```

Note

This program generates 6 random numbers between 1 and MAX (45). Numbers may be repeated. (To get 6 unique numbers, use an array and check duplicates.)

How It Works

srand((unsigned)time(NULL))
Set the seed (use current time)

→

rand()
Generate pseudo-random numbers (0 ~ RAND_MAX)

→

rand() % MAX + 1
Get a number from 1 to MAX (inclusive)

Header Files

<stdlib.h>	<time.h>	<stdio.h>
rand(), srand()	time()	printf()

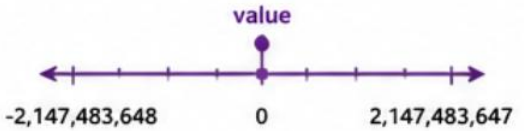
Ch. 9 - Scope, lifetime, and recursion

Functions and Variables

Variable Properties

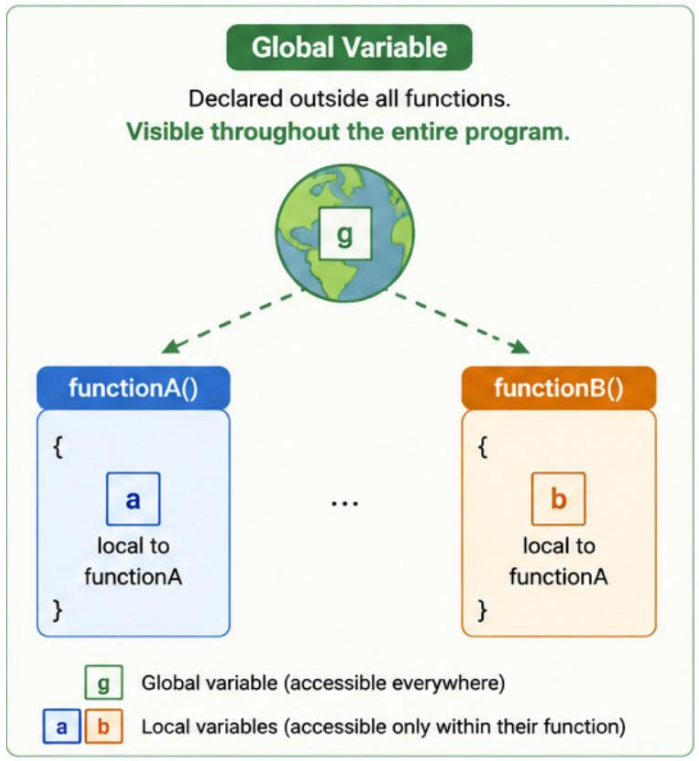
- Variable properties: name, type, size, value + range, lifetime, linkage.
- **Scope**: the **scope** in which a variable is available - its visibility.
- Lifetime: the time it exists in memory.
- Linkage: status of connection with variables in other areas.

- **Variable properties:** **name, type, size, value + range, lifetime, linkage.**

Name	Type	Size	Value + Range	Lifetime	Linkage
An identifier used to refer to the variable. 	The kind of data the variable stores. 	The amount of memory (in bytes) it occupies. 	The current value and the range of possible values. 	The time period during which the variable exists in memory. 	Status of connection with variables in other areas (files). 
Example	**Example**	**Example** (on typical 64-bit system)	**Example (int)**	**Example**	**Example**
count	int	int: 4 bytes double: 8 bytes	value = 10 range: -2,147,483,648 ~ 2,147,483,647	exists from { } block or for entire program	none / internal / external

Scope of Variable: Global & Local Variables

- **Scope** determines where a variable is visible can be use in a program



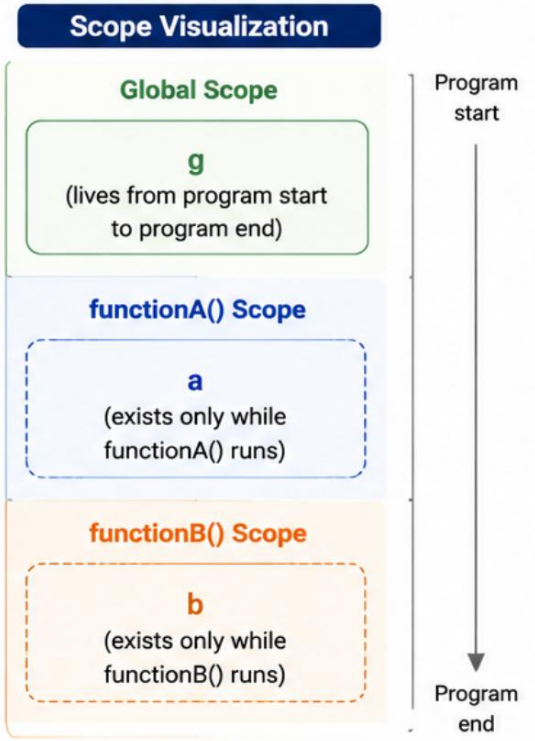

```c
#include <stdio.h>

int g = 10;     // global variable

void functionA(void)
{
   int a = 1;     // local to functionA
    printf("g = %d, a = %d\n", g, a);
}

void functionB(void)
{
   int b = 2;     // local to functionB
    printf("g = %d, b = %d\n", g, b);
}

int main(void)
{
   printf("g = %d\n", g);
   functionA();
   functionB();
   return 0;
}
```



Key Points

- Global variables are visible to all functions.
- Local variables are visible only within the function in which they are declared.
- Local variables are created when the function starts and destroyed when it ends.

Example Output

```
g = 10
g = 10, a = 1
g = 10, b = 2
```

Variable	Scope	Lifetime
g (global)	Entire program	From program start to end
a (local)	functionA() only	From functionA() start to end
b (local)	functionB() only	From functionB() start to end

Local Variables

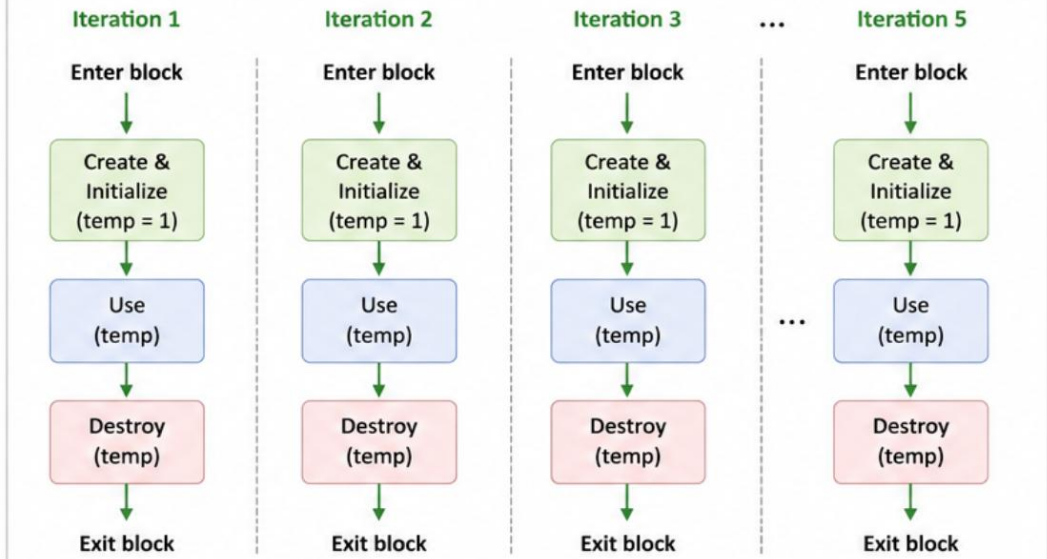
A local variable is declared inside a block or a function.
It is created when the block/function is entered and destroyed when it ends.

Example

```c
#include <stdio.h>
int main(void)
{
    int i;
    for (i = 0; i < 5; i++)
    {
        int temp = 1;  // local variable
        printf("temp = %d\n", temp);
        temp++;
    }
    return 0;
}
```

- Created at the start of each loop iteration
- Initialized with 1
- Exists only inside the loop block
- Destroyed at the end of each iteration

Lifecycle of Local Variable



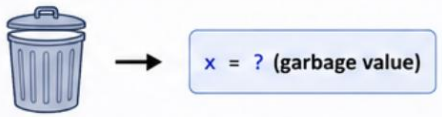
Key Points

- Local variables are declared inside a block or function.
- They can be used only within that block/function (scope).
- The same variable name can be used in different blocks.
- Since it is not initialized, it may have a garbage value.
- Parameters are also a kind of local variable.

Garbage Value (Not Initialized)

A local variable that is not initialized contains an unpredictable (garbage) value.

```c
void func(void)
{
    int x;        // not initialized
    printf("x = %d\n", x);  // garbage value
}
```



Parameters = Local Variable

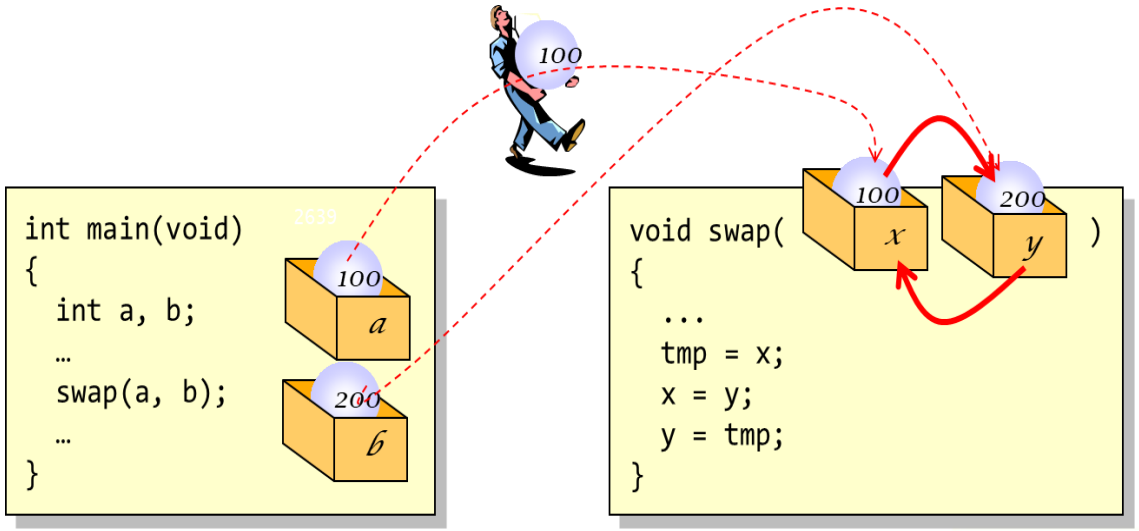
Function parameters are local variables that exist only while the function runs.

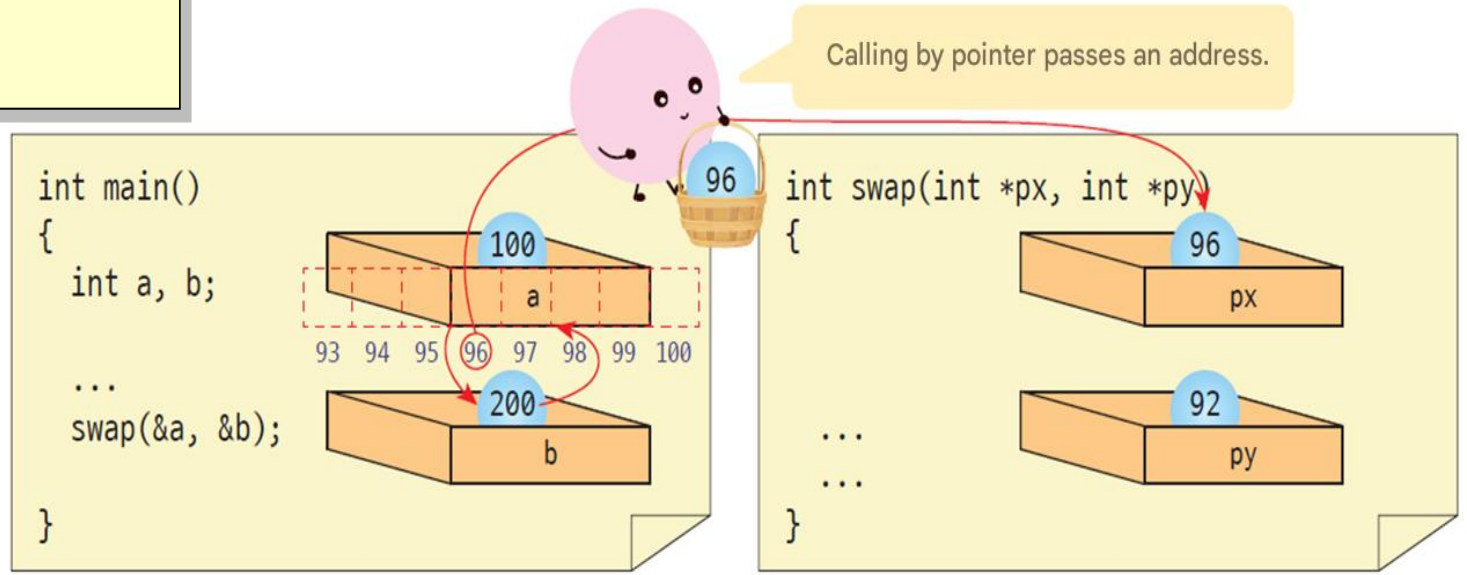
```c
int add(int a, int b)   // a, b are parameters
{
    int sum = a + b;     // local variable
    return sum;          // local variable
}
```

Arguments (from caller) → a, b (parameters) → Local variables inside function

Local Variables: How to pass arguments to parameters (Call by Value vs. Call by Reference)

- Call by value: a copy is passed to the function - the basic method in C.
- Call by reference: the original is passed to the function - in C, this is emulated using pointers.





Function Parameters Are Local Variables

VSC

```c
#include < stdio.h >
void inc ( int counter);

int main( void )
{
    int i ;
    i = 10;
    printf ( "Before calling the function i=%d\n" , i );
    inc ( i );
    printf ( "After calling the function i=%d\n" , i );
    return 0;
}

void inc ( int counter)
{
    counter++;
}
// Before calling the function i=10
// After calling the function i=10   <- unchanged! (call by value)
```

Parameters are also a kind of local variable - they are initialized with the argument values when the function is called.

Global Variables

A global variable is declared outside all functions.
It is created when the program starts and exists until the program ends.

```c
#include <stdio.h>

// Global variable
int g = 10;

void func(void)
{
    // can access global variable
    int x = 5;     // local variable
    g = g + x;
    printf("[func] g = %d, x = %d\n", g, x);
}

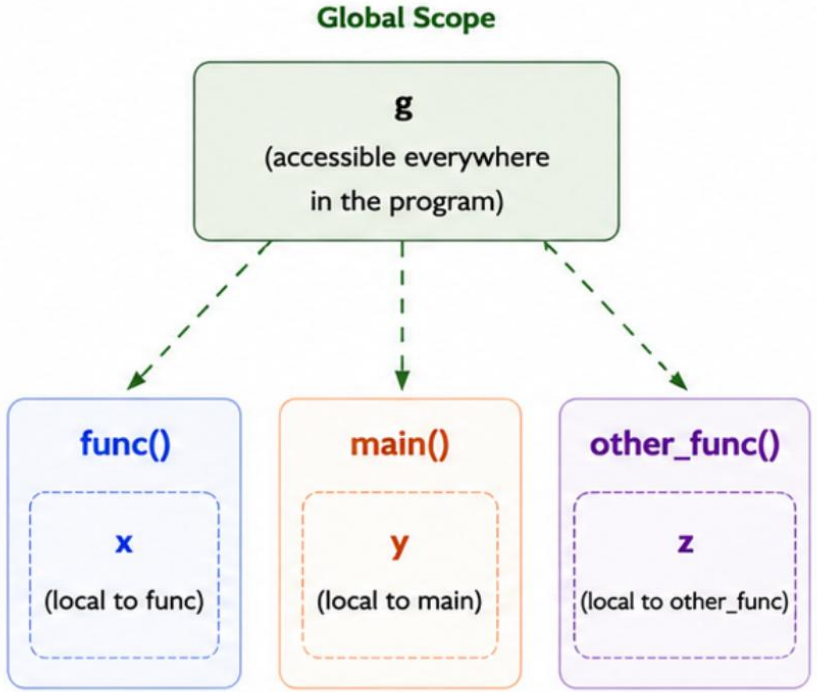
int main(void)
{
    int y = 20;   // local variable
    printf("[main before] g = %d\n", g);
    func();
    printf("[main after]  g = %d, y = %d\n", g, y);
    return 0;
}
```

Global variables are initialized to 0 by the compiler when the program runs.

Global variable g

- Declared outside all functions
- Visible to all functions
- Exists throughout the entire program
- Has a single copy shared by all functions

Scope of Global Variable



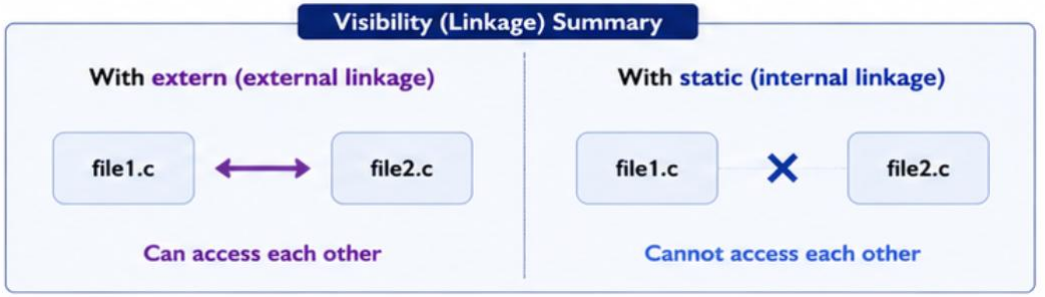
All functions can read / modify the global variable g.

Key Points

- Global variables are declared outside all functions.
- They are **visible** to all functions in the file after their declaration.
- They exist from program start to program end.
- There is only **one copy** of a global variable, shared by all functions.
- Use global variables carefully! (Overuse can make programs hard to understand.)

Linkage & Lifetime of Variables

- In C, extern and static control linkage (visibility to other files) and lifetime (existence time).
- When to use?
  - extern: Use extern when you need to share global variables or functions across multiple files.
  - static: Use static when you want to restrict visibility to the same file or keep a local variable's value.



extern

extern is used to **declare** a variable or function that is **defined in another file.**

Example

file1.c (definition)

```c
#include <stdio.h>
int count = 0;    // global variable defined

void func(void) {
   printf("Hello from func()\n");
}
```

file2.c (use with extern)

```c
#include <stdio.h>
extern int count;          // declaration
extern void func(void);    // declaration
int main(void) {
   count = 10;             // use defined in file1.c
   func();
   printf("count = %d\n", count);
   return 0;
}
```

- The actual storage is in **file1.c**.
- **file2.c** only declares it with **extern** to use it.

static

static limits the visibility (linkage) of a variable or function **to the same file** (internal linkage).

Example

file1.c

```c
#include <stdio.h>
static int count = 0;    // file-local global

static void helper(void) {   // file-local function
   printf("helper()\n");
}

void func(void) {
   count++;          // can use
   helper();         // can use
}
```

file2.c

```c
#include <stdio.h>
extern void func(void);      // OK
extern int count;            // Error!
extern void helper(void);    // Error!

int main(void) {
   func();
   return 0;
}
```

- **static** variables/functions are visible only within **file1.c**.
- Other files cannot access **count** or **helper()**.

What Is Recursion?

- A **function** can also call itself - this is called **recursion**.
- Factorial programming:
  Calculate the factorial (n-1)! by calling the **function** you are currently writing again (recursive call).
- The recursive algorithm consists of a part that recursively calls itself, and a part that stops the recursive call (the base case).

```
#include < stdio.h >

int factorial( int n)
{
    if ( n <= 1 )
        return (1);
    else
        return (n * factorial(n-1) );
}

int main( void )
{
     int x = 0;
     long f;

     printf ( “Enter an integer :” );
     scanf ( “%d”, &n);
     printf (“%d! is %d . \n”, n, factorial(n));

     return 0;
}
```

Factorial Calling Order

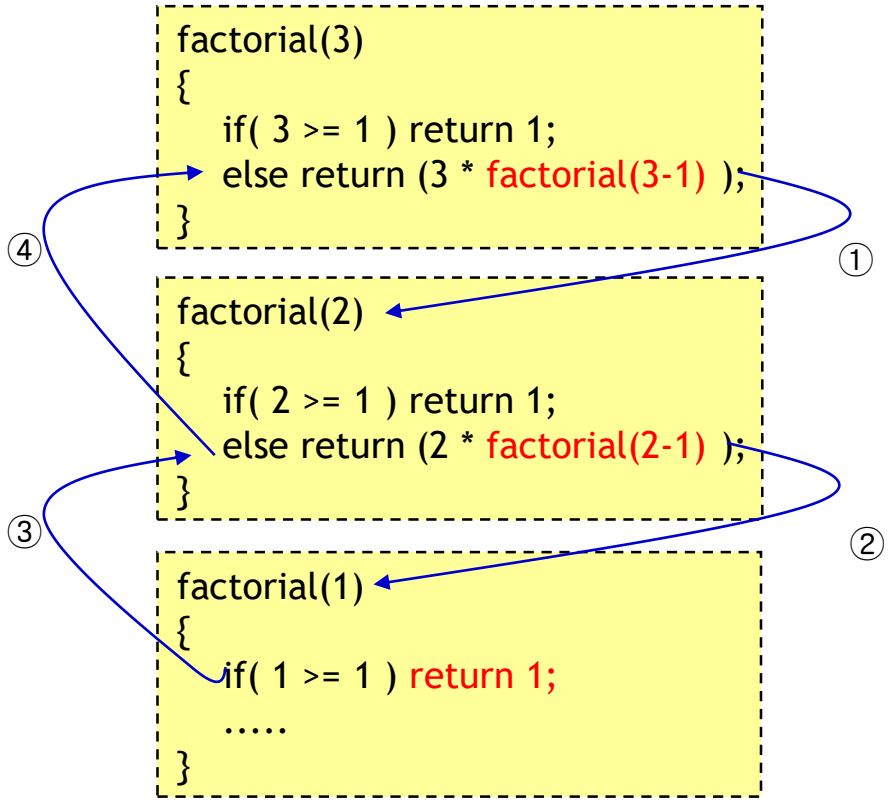
factorial(3) = 3 * factorial(2) ①

= 3 * 2 * factorial(1) ②

= 3 * (2 * 1) ③

= (3 * 2) ④

= 6



Summary: Recursive Function

Recursive Function

A recursive function is a function that calls itself to solve a problem by breaking it into smaller subproblems.

Example: Factorial Function in C

```c
int factorial(int n)
{
   if (n <= 1)
      return 1;          // Base case

   return n * factorial(n - 1);  // Recursive call
}
```

Two Essential Parts of Recursion

1) Base case (Stopping condition)

The condition that stops further recursive calls and returns a result.

```c
if (n <= 1) return 1;
```

2) Recursive call

The part that calls the function itself with a smaller input (closer to the base case).

```c
return n * factorial(n - 1);
```

How factorial(3) Works

The function calls itself with smaller values until the base case is reached.

Call Sequence

- **Step 1** — `factorial(3)`: if (3 <= 1)? No; return 3 * factorial(2)
- **Step 2** — `factorial(2)`: if (2 <= 1)? No; return 2 * factorial(1)
- **Step 3** — `factorial(1)`: if (1 <= 1)? Yes; return 1 (base case)
- **Step 4** — Return 1 to previous call

Return (Unwinding)

- factorial(1) returned 1
- factorial(2) returns 2 * 1 = 2
- factorial(3) returns 3 * 2 = 6
- **Final Result** — factorial(3) = 6

Key Takeaways

- A recursive function must have a base case and a recursive call.
- Each call reduces the problem size and moves closer to the base case.
- When the base case is reached, the results are returned back step by step.

Example:

factorial(5) = 5 × 4 × 3 × 2 × 1 = 120

Q & A

Lab & Quiz: right after this - see you in the practice session!